

# هوش مصنوعی و کارگاه

دانشکده ریاضی و علوم کامپیوتر دانشگاه صنعتی امیرکبیر



مدرس کلاس: مهدی قطعی

استاد گروه علوم کامپیوتر

[www.aut.ac.ir](http://www.aut.ac.ir)

مدرس کارگاه: بهنام یوسفی مهر

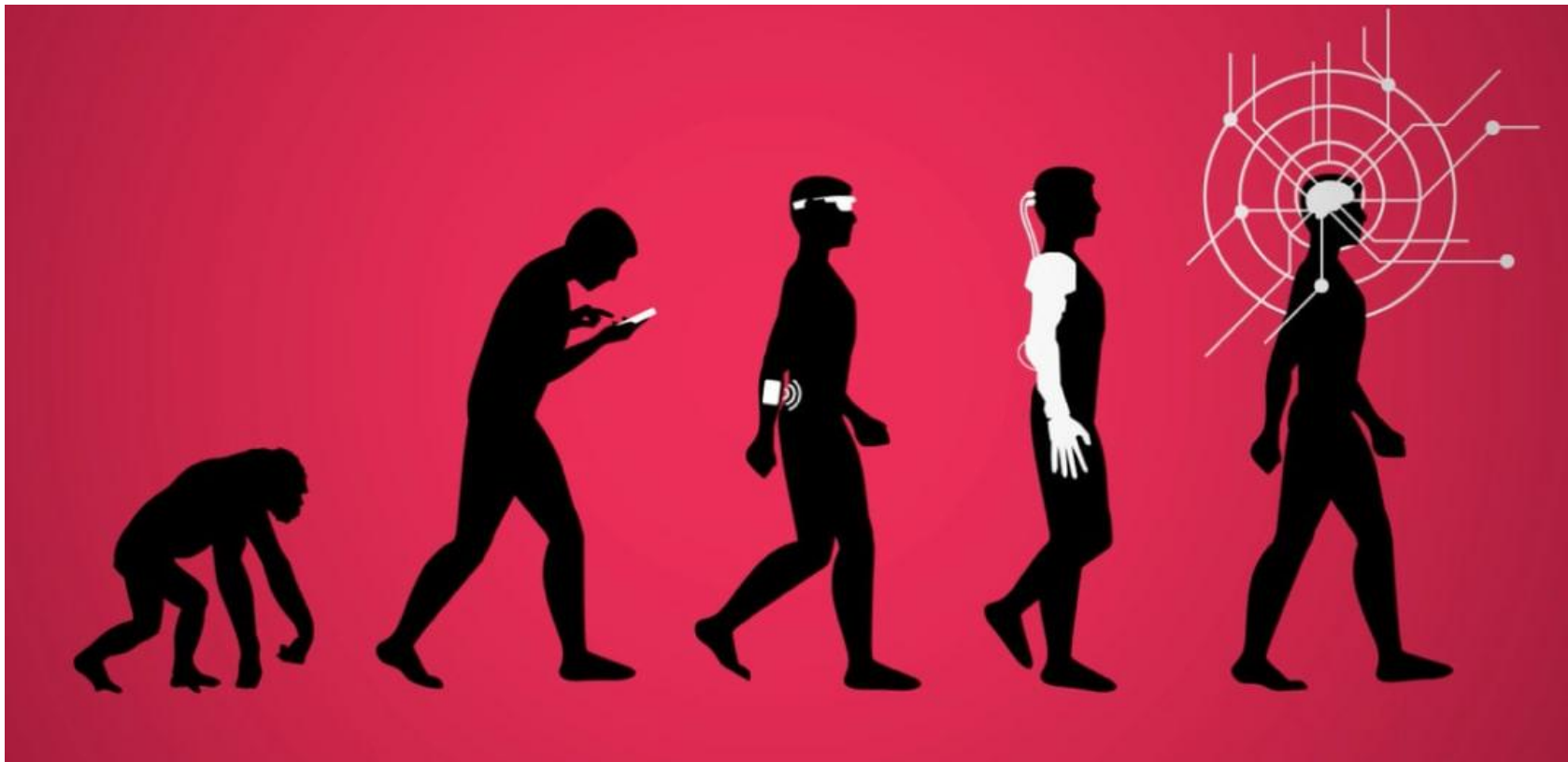
دانشجوی دکتری علوم کامپیوتر

# Artificial Intelligence & Workshop

---

- Evolutionary Algorithm

# What is an Evolutionary Algorithm?



# Contents

---

- Recap of Evolutionary Metaphor
- Basic scheme of an EA
- Basic Components:
  - Representation / Evaluation / Population / Parent Selection / Recombination / Mutation / Survivor Selection / Termination
- Examples : eight queens / knapsack
- Typical behaviours of EAs
- EC in context of global optimisation

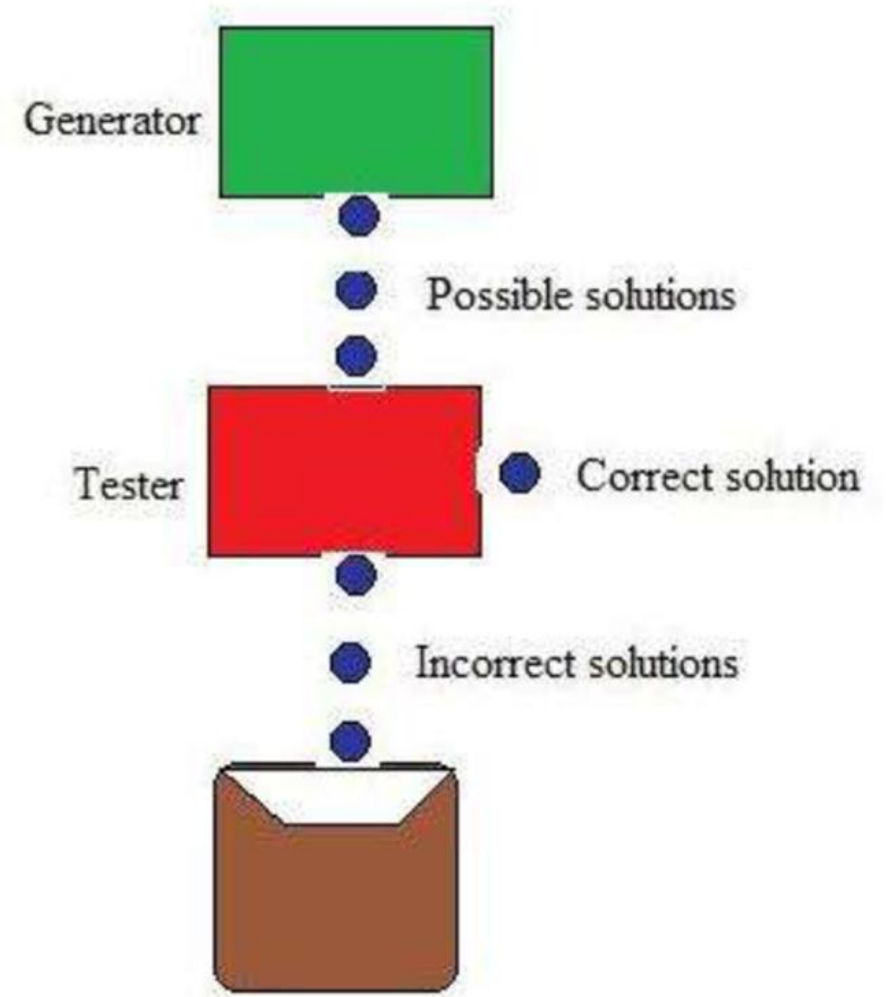
# Recap of EC metaphor

- A population of individuals exists in an environment with limited resources
- *Competition* for those resources causes selection of those *fitter* individuals that are better adapted to the environment
- These individuals act as seeds for the generation of new individuals through recombination and mutation
- The new individuals have their fitness evaluated and compete (possibly also with parents) for survival.
- Over time *Natural selection* causes a rise in the fitness of the population

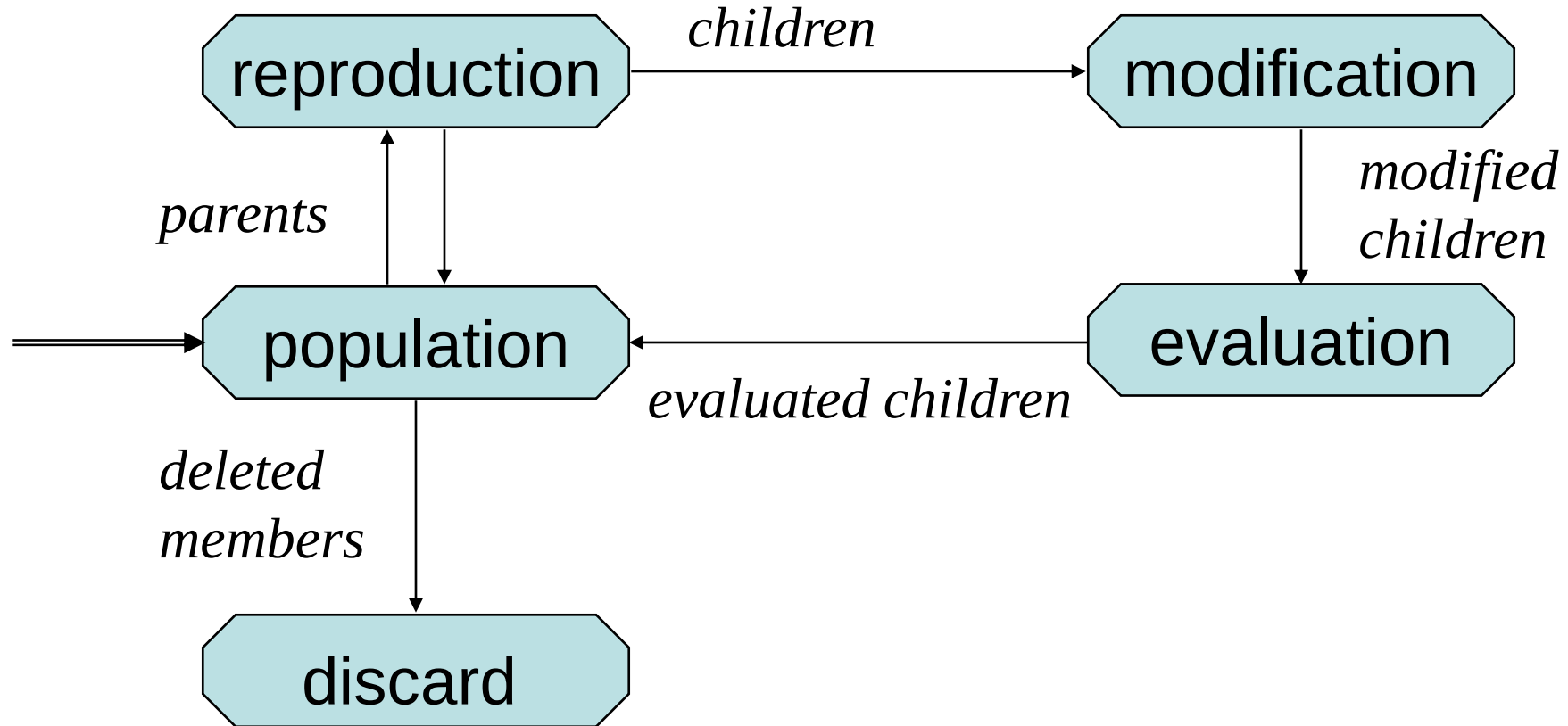


# Recap 2:

- EAs fall into the category of “generate and test” algorithms
- They are stochastic, population-based algorithms
- Variation operators (recombination and mutation) create the necessary diversity and thereby facilitate novelty
- Selection reduces diversity and acts as a force pushing quality



# General Scheme of EAs



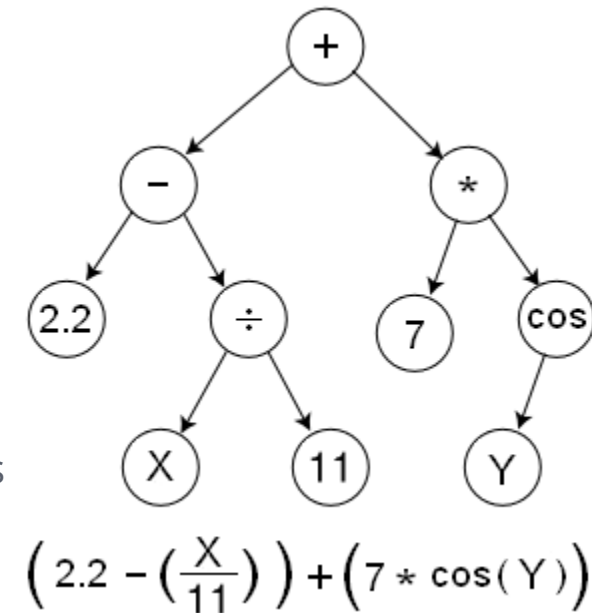
# Pseudocode of EA

---

- START
- Generate the initial population
- Compute fitness
  - REPEAT
    - Selection
    - Crossover
    - Mutation
    - Compute fitness
  - UNTIL population has converged
- STOP

# What are the different types of EAs

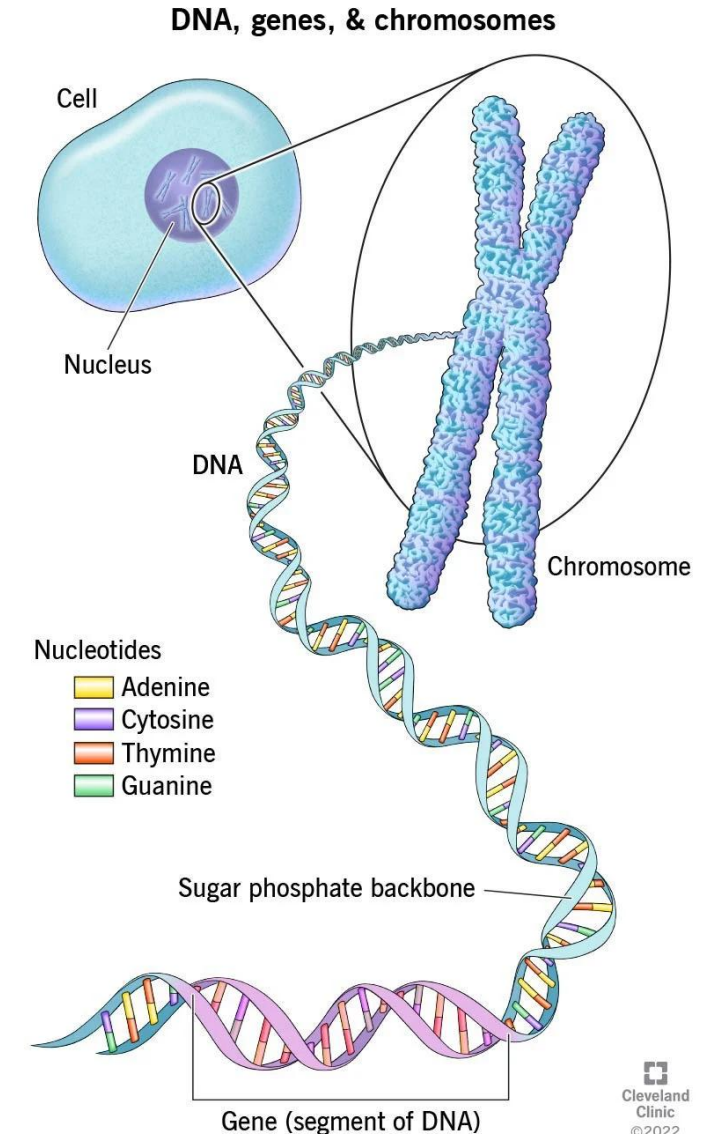
- Historically different flavours of EAs have been associated with different representations
  - Binary strings : Genetic Algorithms
  - Real-valued vectors : Evolution Strategies
  - Finite state Machines: Evolutionary Programming
  - LISP trees: Genetic Programming
    - In tree-based GP, the computer programs are represented in tree structures that are evaluated recursively to produce the resulting multivariate expressions. Traditional nomenclature states that a *tree node* (or just node) is an operator  $[+, -, *, /]$  and a *terminal node* (or leaf) is a variable.
    - Lisp was the first programming language applied to tree-based GP, as the structure of this language matches the structure of the trees. However, many other languages such as Python, Java, and C++ have been used to develop tree-based GP applications.
- These differences are largely irrelevant, best strategy
  - choose representation to suit problem
  - choose variation operators to suit representation
- Selection operators only use fitness and so are independent of representation



# Representations

- Candidate solutions (**individuals**) exist in *phenotype* space
- They are encoded in **chromosomes**, which exist in *genotype* space
  - Encoding : phenotype=> genotype (not necessarily one to one)
  - Decoding : genotype=> phenotype (must be one to one)
- Chromosomes contain **genes**, which are in (usually fixed) positions called **loci** (sing. locus) and have a value (**allele**)

In order to find the global optimum, every feasible solution must be represented in genotype space



# Representations

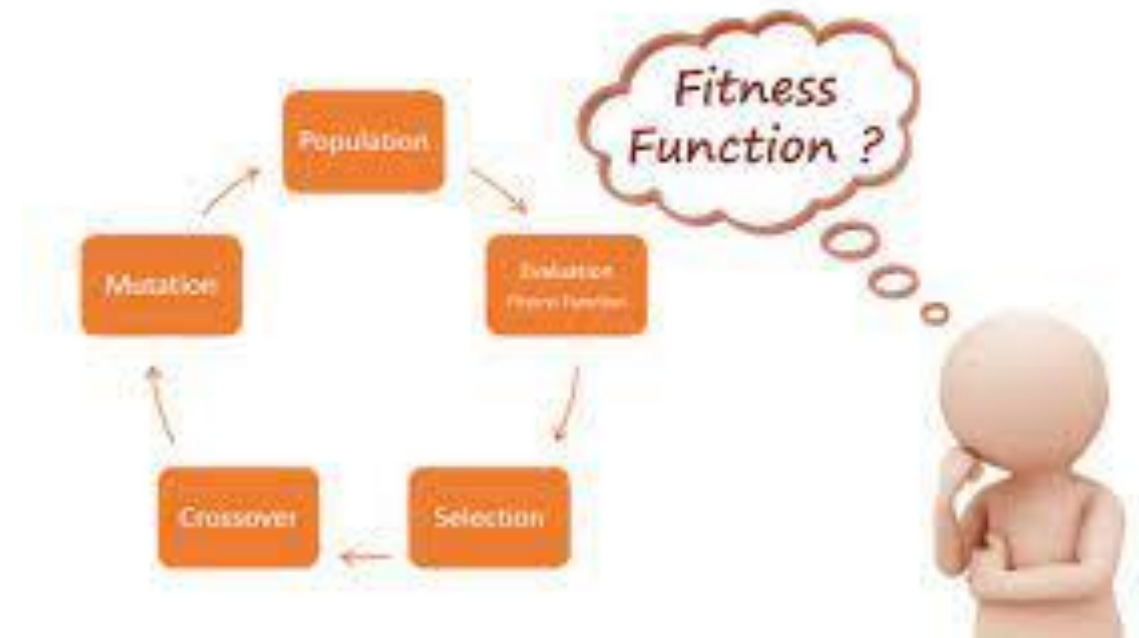
|    |   |   |   |   |   |   |            |
|----|---|---|---|---|---|---|------------|
| A1 | 0 | 0 | 0 | 0 | 0 | 0 | Gene       |
| A2 | 1 | 1 | 1 | 1 | 1 | 1 | Chromosome |
| A3 | 1 | 0 | 1 | 0 | 1 | 1 |            |
| A4 | 1 | 1 | 0 | 1 | 1 | 0 | Population |

|              |                   |
|--------------|-------------------|
| Chromosome A | 1 5 3 2 6 4 7 9 8 |
| Chromosome B | 8 5 6 7 2 3 1 4 9 |

|              |                                            |
|--------------|--------------------------------------------|
| Chromosome A | 1.235 5.323 0.454 2.321 2.454              |
| Chromosome B | (left), (back), (left), (right), (forward) |

# Evaluation (Fitness) Function

- Represents the requirements that the population should adapt to
- a.k.a. *quality* function or *objective* function
- Assigns a single real-valued fitness to each phenotype which forms the basis for selection
  - So the more discrimination (different values) the better
- Typically we talk about fitness being maximised
  - Some problems may be best posed as minimisation problems, but conversion is trivial



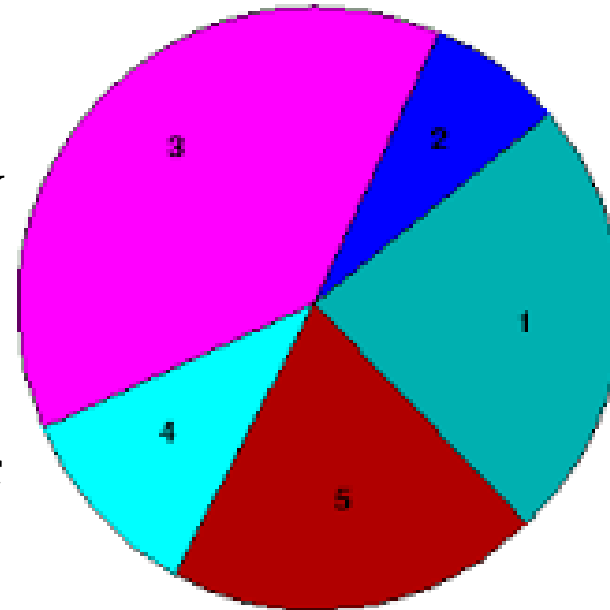
# Population

- Holds (representations of) possible solutions
- Usually has a fixed size and is a *multiset* of genotypes
- Some sophisticated EAs also assert a spatial structure on the population e.g., a grid.
- Selection operators usually take whole population into account i.e., reproductive probabilities are *relative* to *current* generation
- **Diversity** of a population refers to the number of different fitnesses / phenotypes / genotypes present (note not the same thing)



# Parent Selection Mechanism

- Assigns variable probabilities of individuals acting as parents depending on their fitnesses
- Usually probabilistic
  - high quality solutions more likely to become parents than low quality
  - but not guaranteed
  - even worst in current population usually has non-zero probability of becoming a parent
- This *stochastic* nature can aid escape from local optima



| Population | Fitness |
|------------|---------|
| 1          | 25.0    |
| 2          | 5.0     |
| 3          | 40.0    |
| 4          | 10.0    |
| 5          | 20.0    |

# Variation Operators

- Role is to generate new candidate solutions
- Usually divided into two types according to their **arity** (number of inputs):
  - Arity 1 : mutation operators
  - Arity >1 : Recombination operators
  - Arity = 2 typically called **crossover**
- There has been much debate about relative importance of recombination and mutation
  - Nowadays most EAs use both
  - Choice of particular variation operators is representation dependant



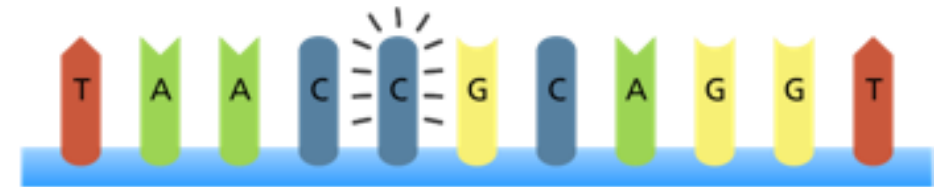
# Mutation

- Acts on one genotype and delivers another
- Element of randomness is essential and differentiates it from other unary heuristic operators
- Importance ascribed depends on representation and dialect:
  - Binary GAs – background operator responsible for preserving and introducing diversity
  - Continuous variables – only search operator
  - GP – hardly used
- May guarantee connectedness of search space and hence convergence proofs

Original sequence



Point mutation



# Mutation: Local Modification

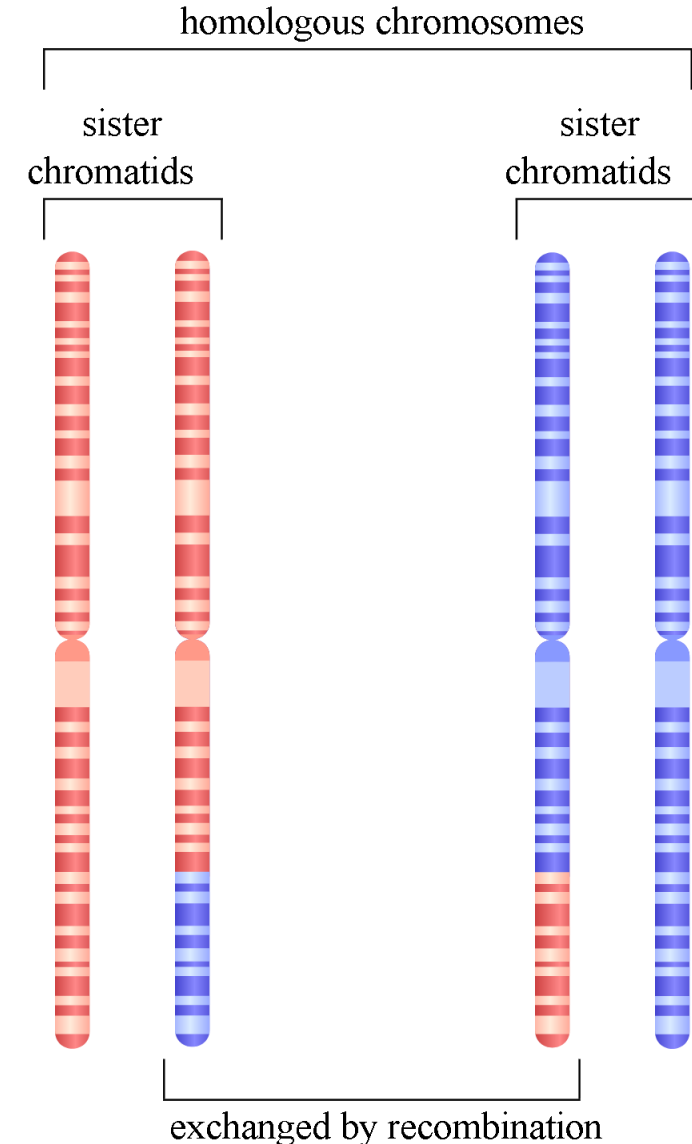
Before: (1 0 1 1 0 1 1 0)  
After: (0 1 1 0 0 1 1 0)

Before: (1.38 -69.4 326.44 0.1)  
After: (1.38 -67.5 326.44 0.1)

- Causes movement in the search space
- Restores lost information to the population

# Recombination

- Merges information from parents into offspring
- Choice of what information to merge is stochastic
- Most offspring may be worse, or the same as the parents
- Hope is that some are better by combining elements of genotypes that lead to good traits
- Principle has been used for millennia by breeders of plants and livestock



# Crossover: Recombination

---

P1 (0 1 **1 0** 1 0 0 0) → (0 1 0 1 1 0 0 0) C1  
P2 (1 1 **0 1** 1 0 1 0) → (1 1 1 0 1 0 1 0) C2

Crossover is a critical feature of genetic algorithms:

- It greatly accelerates search early in evolution of a population
- It leads to effective combination of schemata (subsolutions on different chromosomes)

# Crossover on real-value

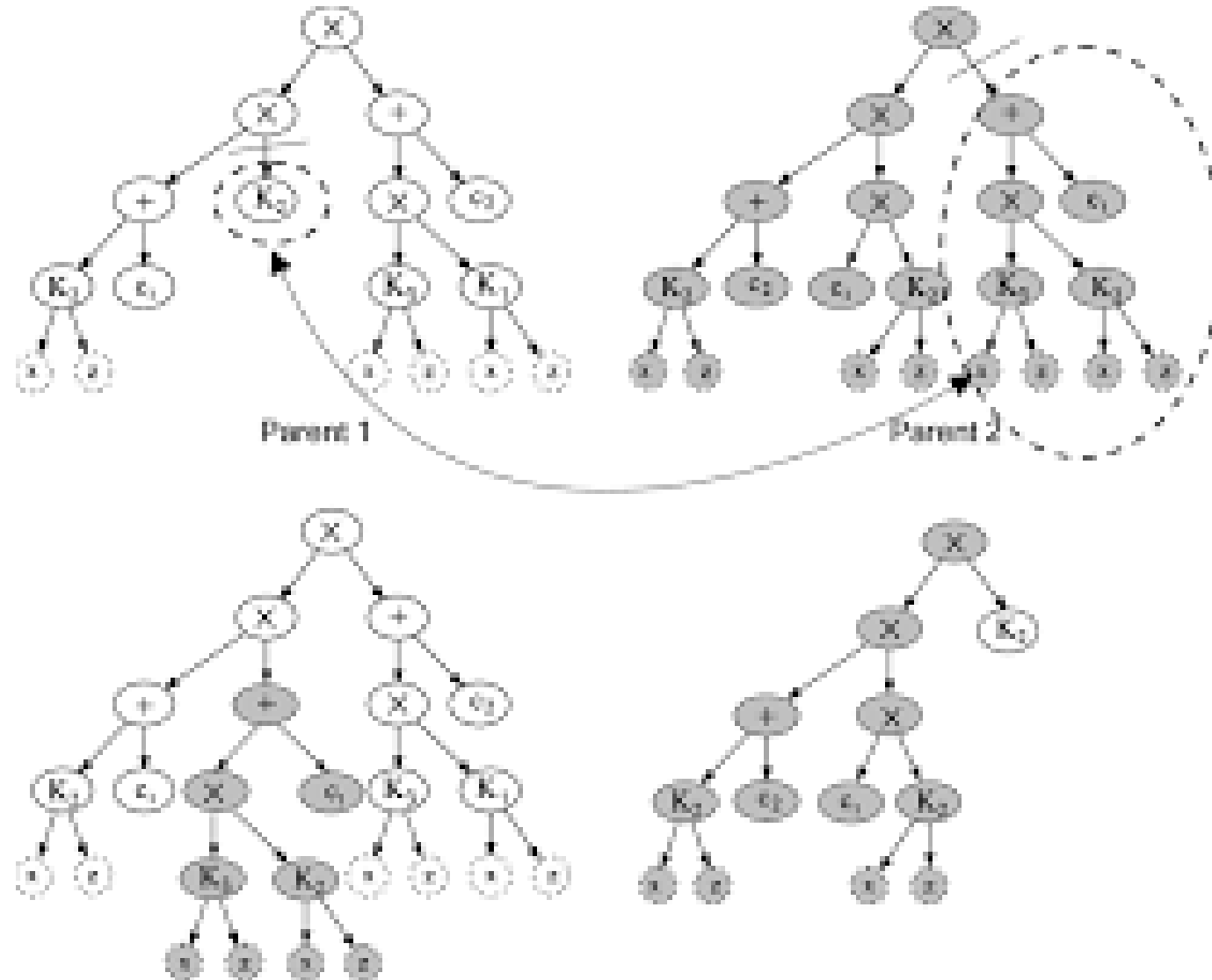
---

- Let  $x$  and  $y$  belong to  $\mathbb{R}^n$  and  $\lambda \in (0,1)$
- The offspring can be defined as:

$$\lambda x + (1 - \lambda)y$$

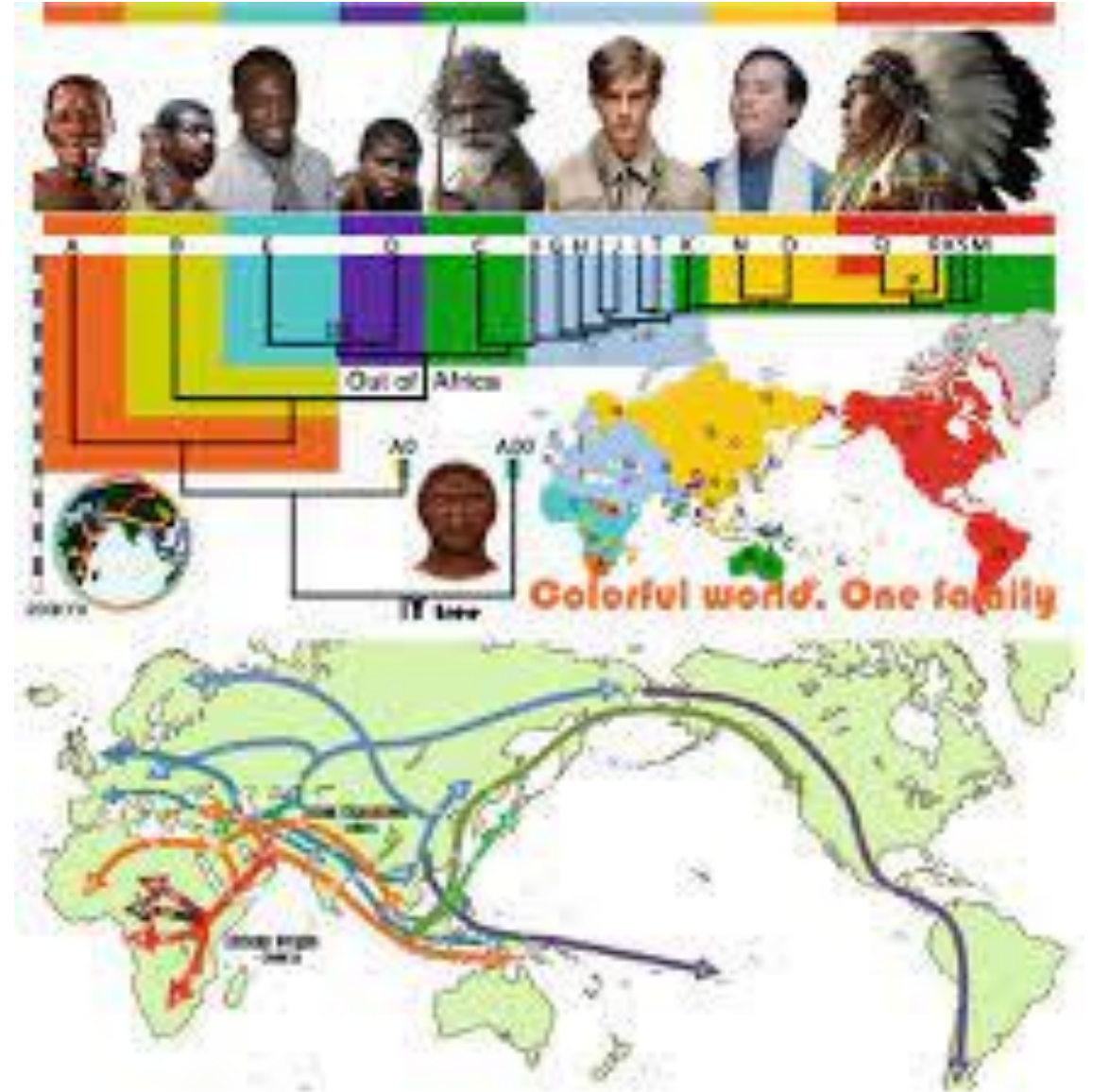
$$\lambda y + (1 - \lambda)x$$

# Crossover on graphs



# Survivor Selection

- a.k.a. *replacement*
- Most EAs use fixed population size so need a way of going from (parents + offspring) to next generation
- Often deterministic
  - Fitness based : e.g., rank parents+offspring and take best
  - Age based: make as many offspring as parents and delete all parents
- Sometimes do combination (elitism)



# Initialisation / Termination

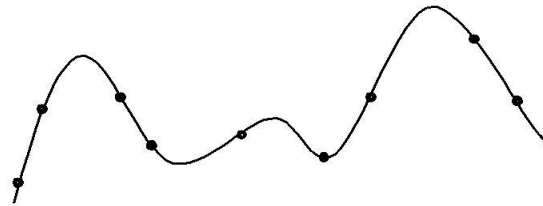
---

- Initialisation usually done at random,
  - Need to ensure even spread and mixture of possible allele values
  - Can include existing solutions, or use problem-specific heuristics, to “seed” the population
- Termination condition checked every generation
  - Reaching some (known/hoped for) fitness
  - Reaching some maximum allowed number of generations
  - Reaching some minimum level of diversity
  - Reaching some specified number of generations without fitness improvement

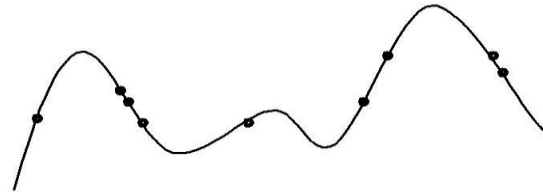
# Typical behaviour of an EA

---

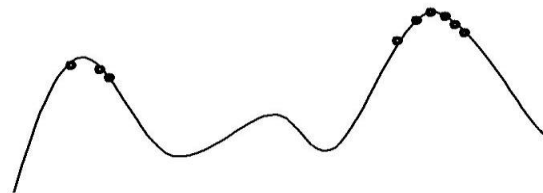
## Phases in optimising on a 1-dimensional fitness landscape



Early phase:  
quasi-random population distribution



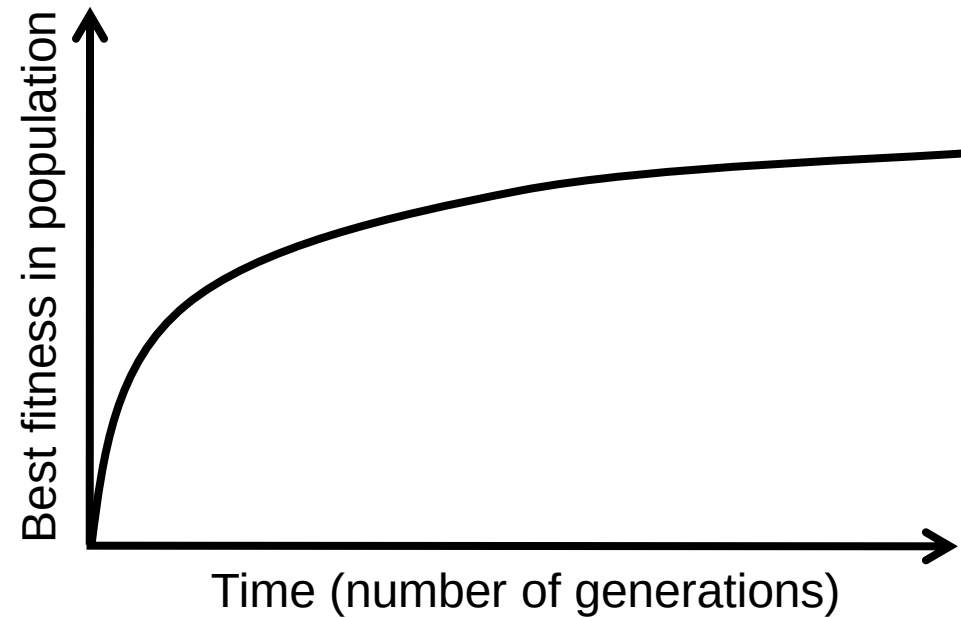
Mid-phase:  
population arranged around/on hills



Late phase:  
population concentrated on high hills

# Typical run: progression of fitness

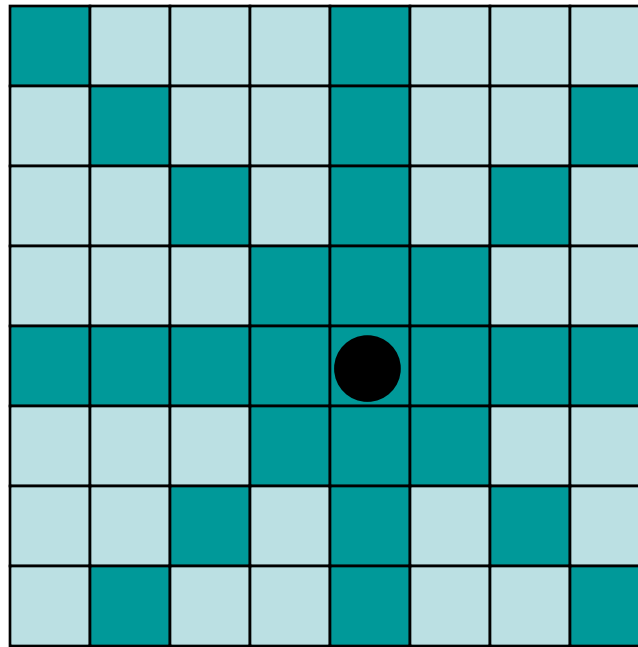
---



Typical run of an EA shows so-called “anytime behavior”

# Example: the 8 queens problem

---



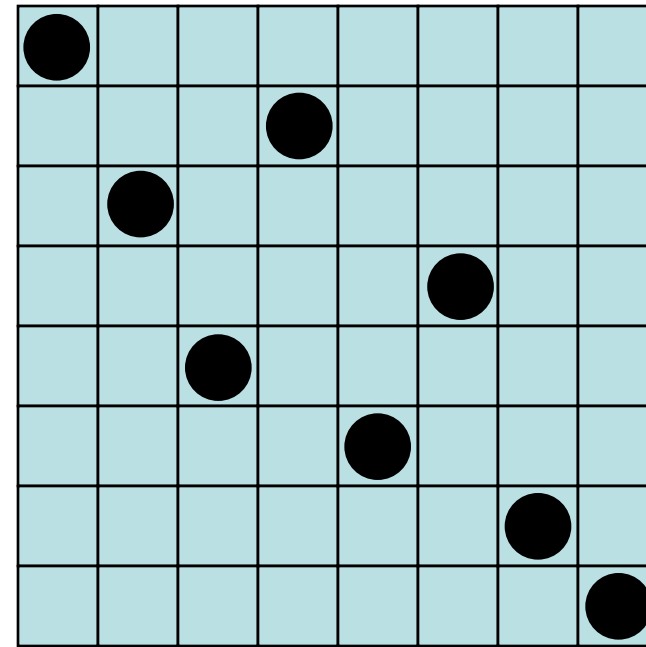
Place 8 queens on an 8x8 chessboard in such a way that they cannot check each other

# The 8 queens problem:

## Representation

Phenotype:  
a board configuration

Genotype:  
a permutation of  
the numbers 1 - 8



Obvious mapping

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 5 | 2 | 6 | 4 | 7 | 8 |
|---|---|---|---|---|---|---|---|

# 8 Queens Problem: Fitness evaluation

---

- Penalty of one queen:  
the number of queens she can check.
- Penalty of a configuration:  
the sum of the penalties of all queens.
- Note: penalty is to be minimized
- Fitness of a configuration:  
inverse penalty to be maximized

# The 8 queens problem: Mutation

---

Small variation in one permutation, e.g.:

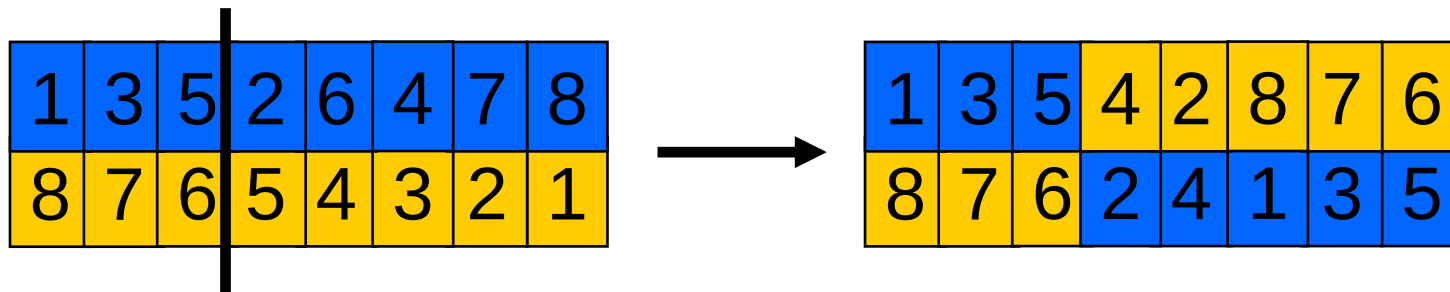
- swapping values of two randomly chosen positions,



# The 8 queens problem: Recombination

Combining two permutations into two new permutations:

- choose random crossover point
- copy first parts into children
- create second part by inserting values from other parent:
  - in the order they appear there
  - beginning after crossover point
  - skipping values already in child



# The 8 queens problem: Selection

---

- Parent selection:
  - Pick 5 parents and take best two to undergo crossover
- Survivor selection (replacement)
  - When inserting a new child into the population, choose an existing member to replace by:
  - sorting the whole population by decreasing fitness
  - enumerating this list from high to low
  - replacing the first with a fitness lower than the given child

# Issues for GA Practitioners

---

- Choosing basic implementation issues:
  - representation
  - population size, mutation rate, ...
  - selection, deletion policies
  - crossover, mutation operators
- Termination Criteria
- Performance, scalability
- Solution is only as good as the evaluation function (often hardest part)

# 8 Queens Problem: summary

|                           |                                       |
|---------------------------|---------------------------------------|
| Representation            | Permutations                          |
| Recombination             | “Cut-and-crossfill” crossover         |
| Recombination probability | 100%                                  |
| Mutation                  | Swap                                  |
| Mutation probability      | 80%                                   |
| Parent selection          | Best 2 out of random 5                |
| Survival selection        | Replace worst                         |
| Population size           | 100                                   |
| Number of Offspring       | 2                                     |
| Initialisation            | Random                                |
| Termination condition     | Solution or 10,000 fitness evaluation |

Note that it is ***only one possible***  
set of choices of operators and parameters

# The Traveling Salesman Problem

---

Find a tour of a given set of cities so that

- each city is visited only once
- the total distance traveled is minimized

# Representation

---

Representation is an ordered list of city numbers known as an *order-based* GA.

|           |              |            |             |
|-----------|--------------|------------|-------------|
| 1) London | 3) Dunedin   | 5) Beijing | 7) Tokyo    |
| 2) Venice | 4) Singapore | 6) Phoenix | 8) Victoria |

CityList1    (3  5  7  2  1  6  4  8)

CityList2    (2  5  7  6  8  1  3  4)

# Crossover

---

Crossover combines inversion and recombination:

|         |   |   |   |   |   |   |   |      |
|---------|---|---|---|---|---|---|---|------|
|         |   | * |   | * |   |   |   |      |
| Parent1 | ( | 3 | 5 | 7 | 2 | 1 | 6 | 4 8) |
| Parent2 | ( | 2 | 5 | 7 | 6 | 8 | 1 | 3 4) |
| Child   | ( | 2 | 5 | 7 | 1 | 6 | 8 | 3 4) |

This operator is called the *Order1* crossover.

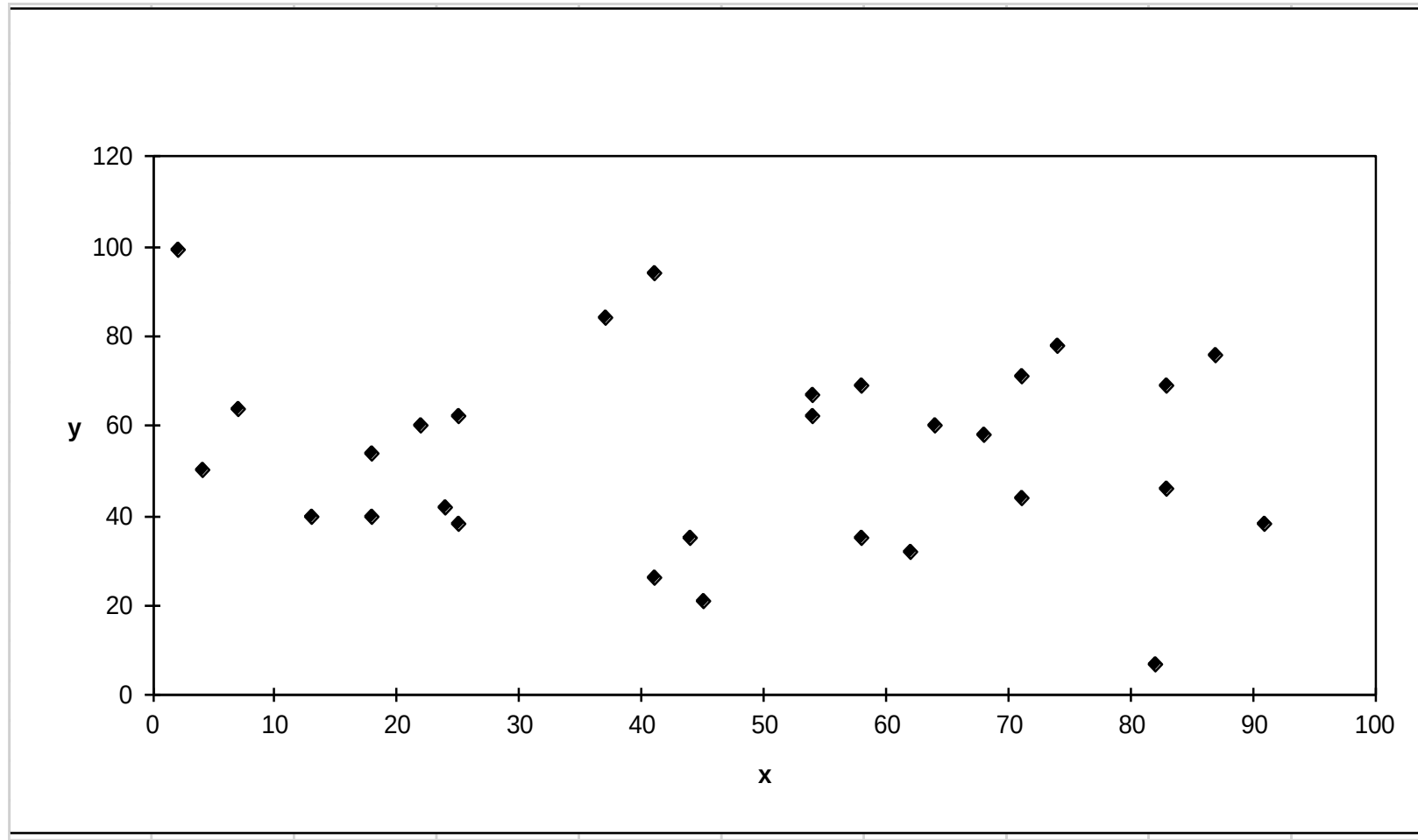
# Mutation

---

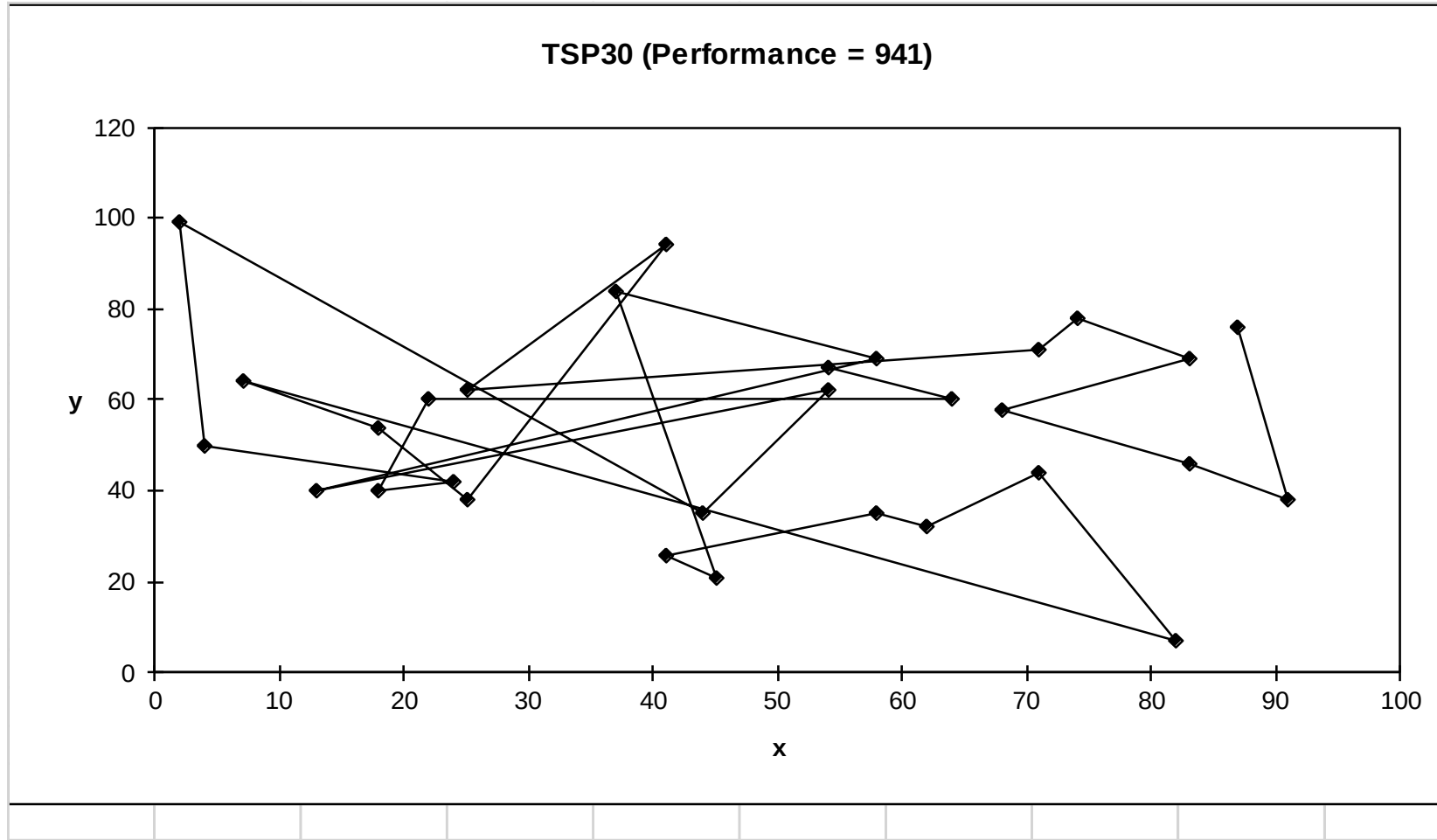
Mutation involves reordering of the list:

|         |   |   |   |   |   |   |   |   |    |
|---------|---|---|---|---|---|---|---|---|----|
|         |   |   | * |   | * |   |   |   |    |
| Before: | ( | 5 | 8 | 7 | 2 | 1 | 6 | 3 | 4) |
| After:  | ( | 5 | 8 | 6 | 2 | 1 | 7 | 3 | 4) |

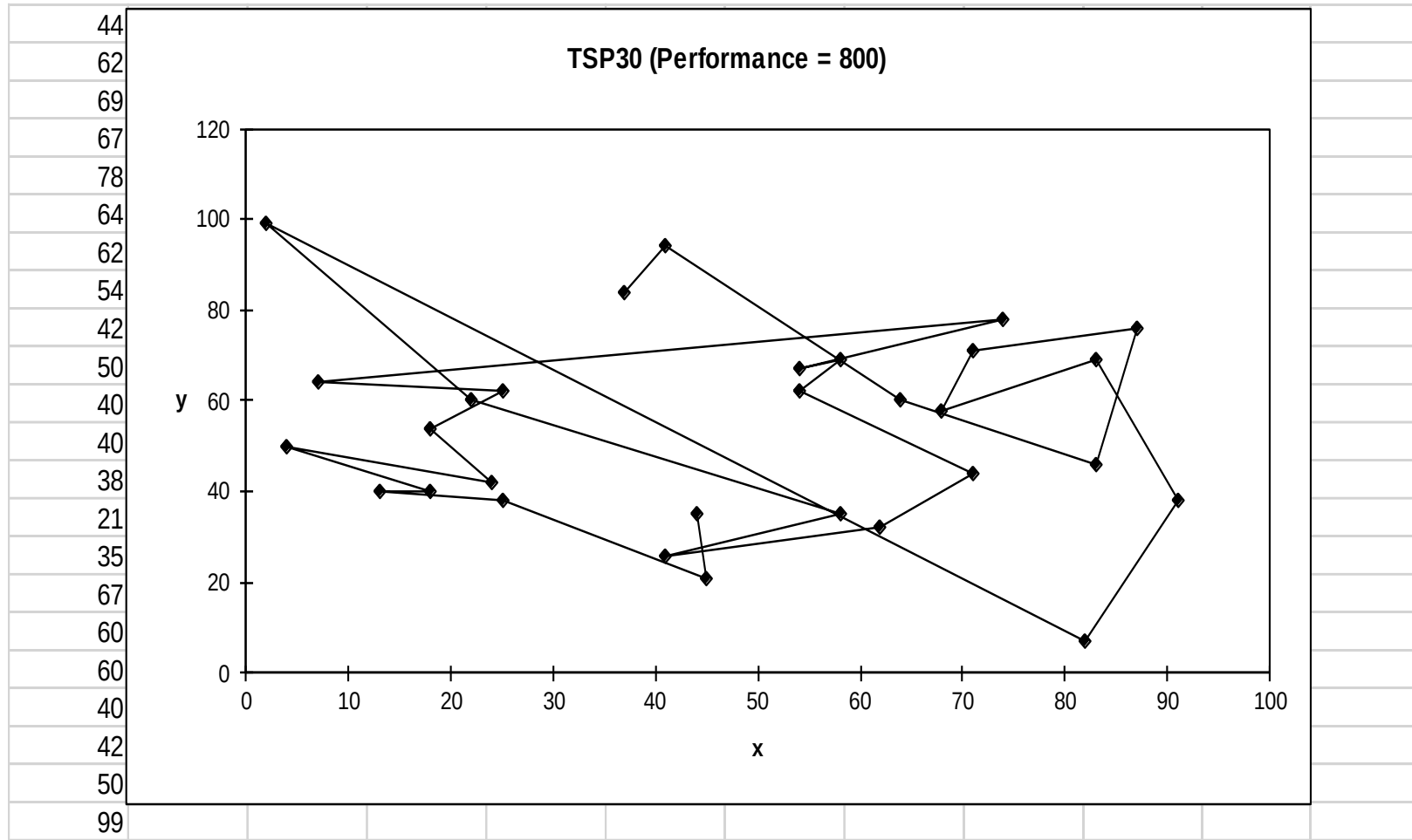
# TSP Example: 30 Cities



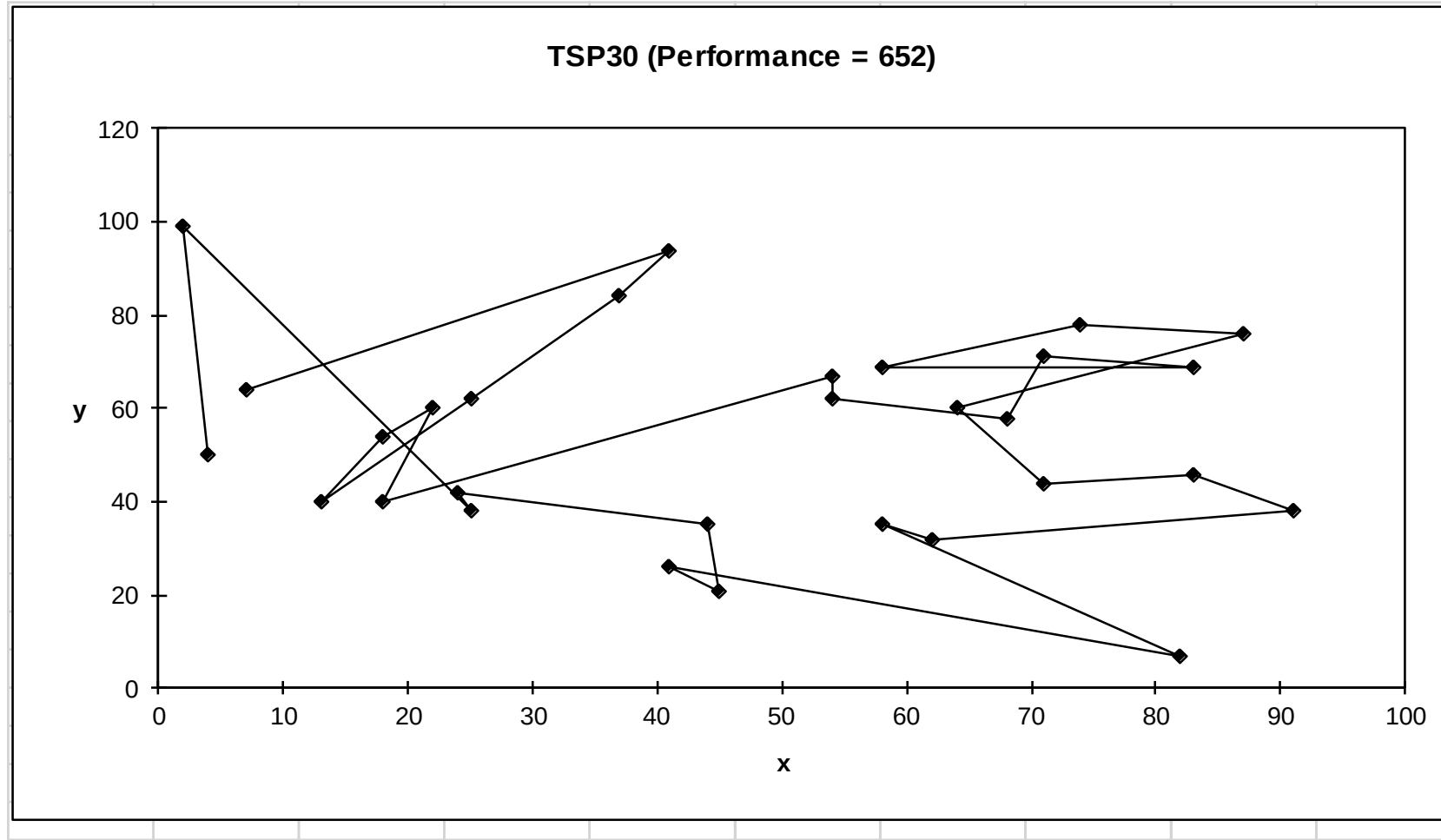
# Solution $i$ (Distance = 941)



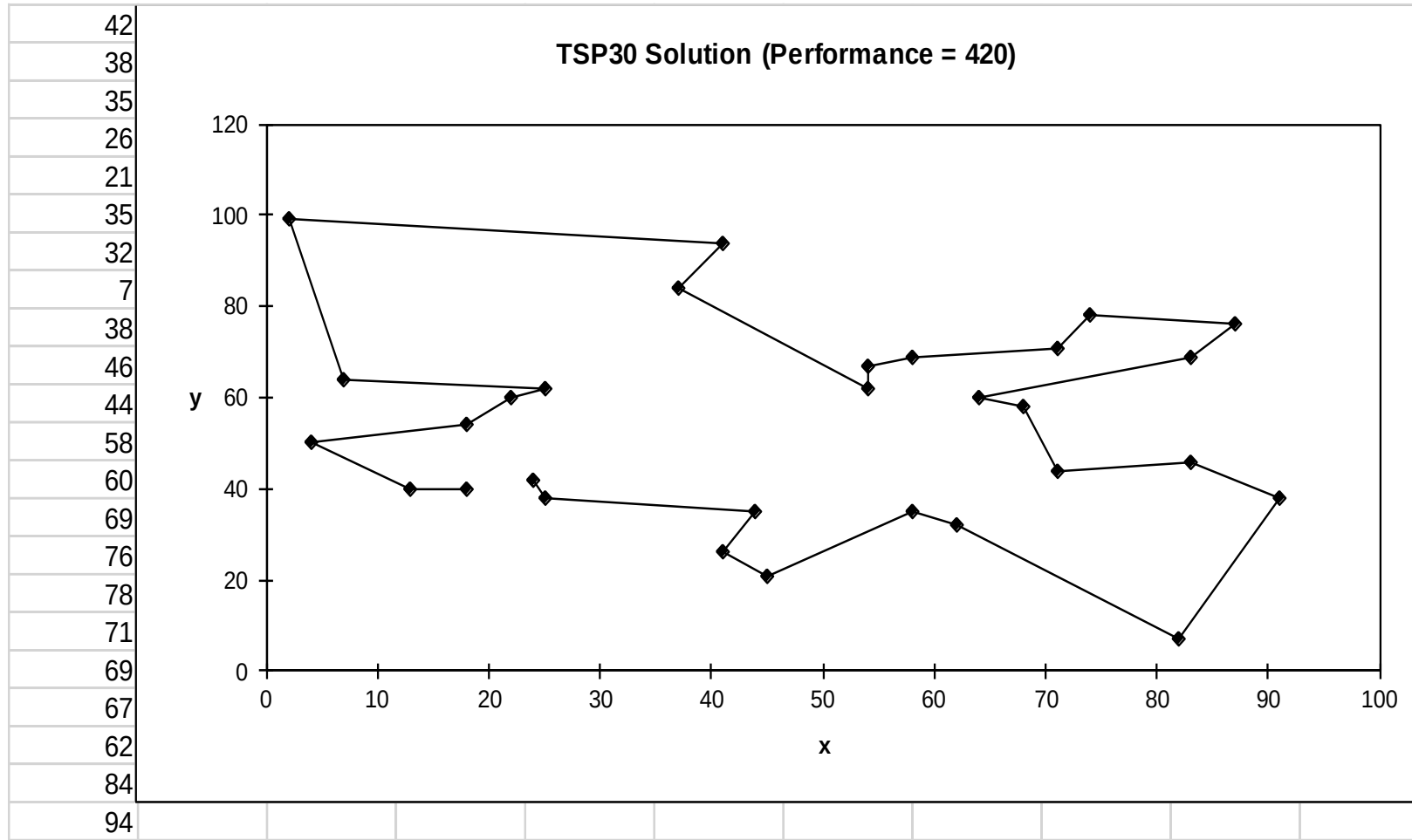
# Solution $j$ (Distance = 800)



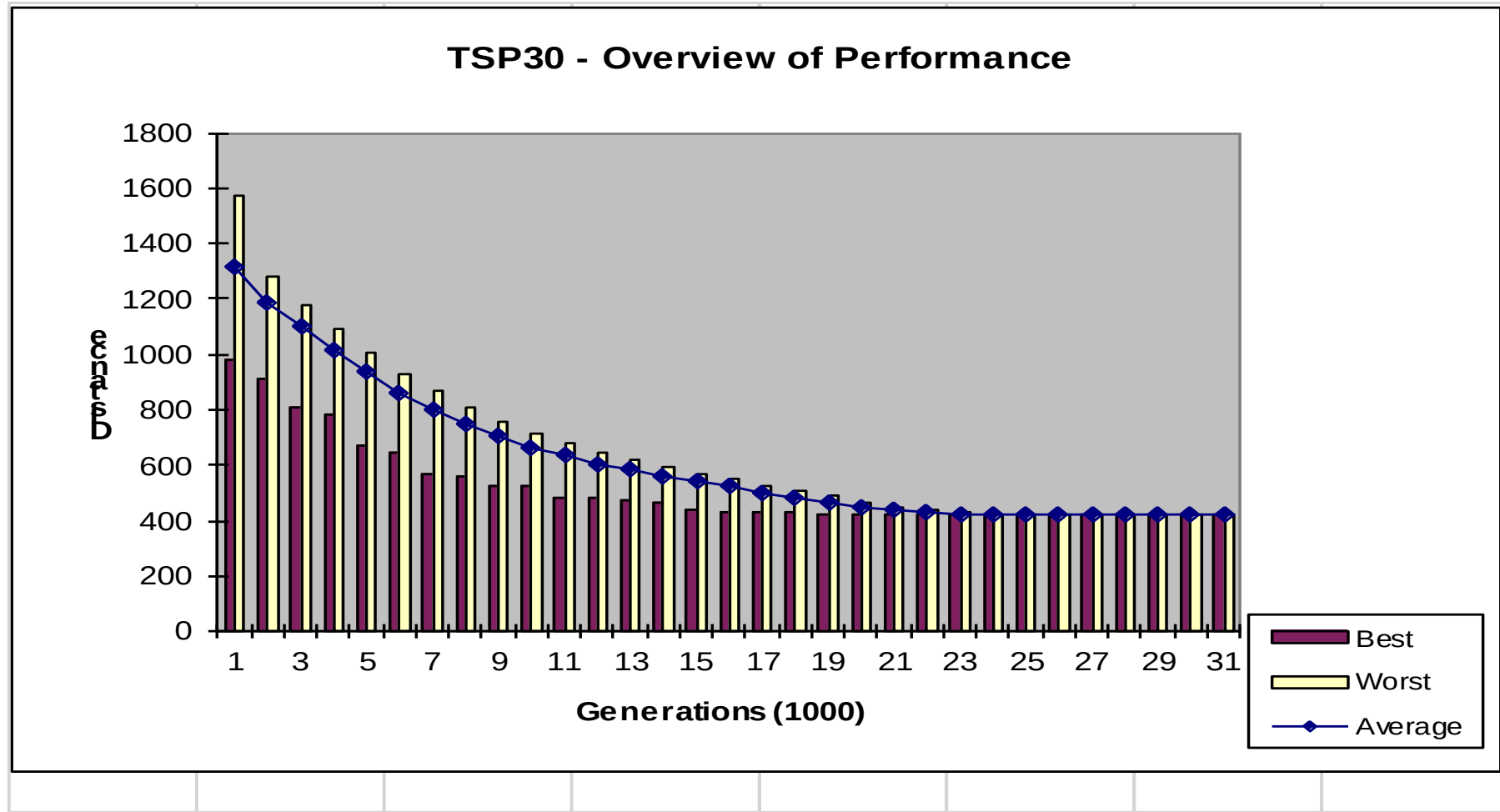
# Solution<sub>k</sub>(Distance = 652)



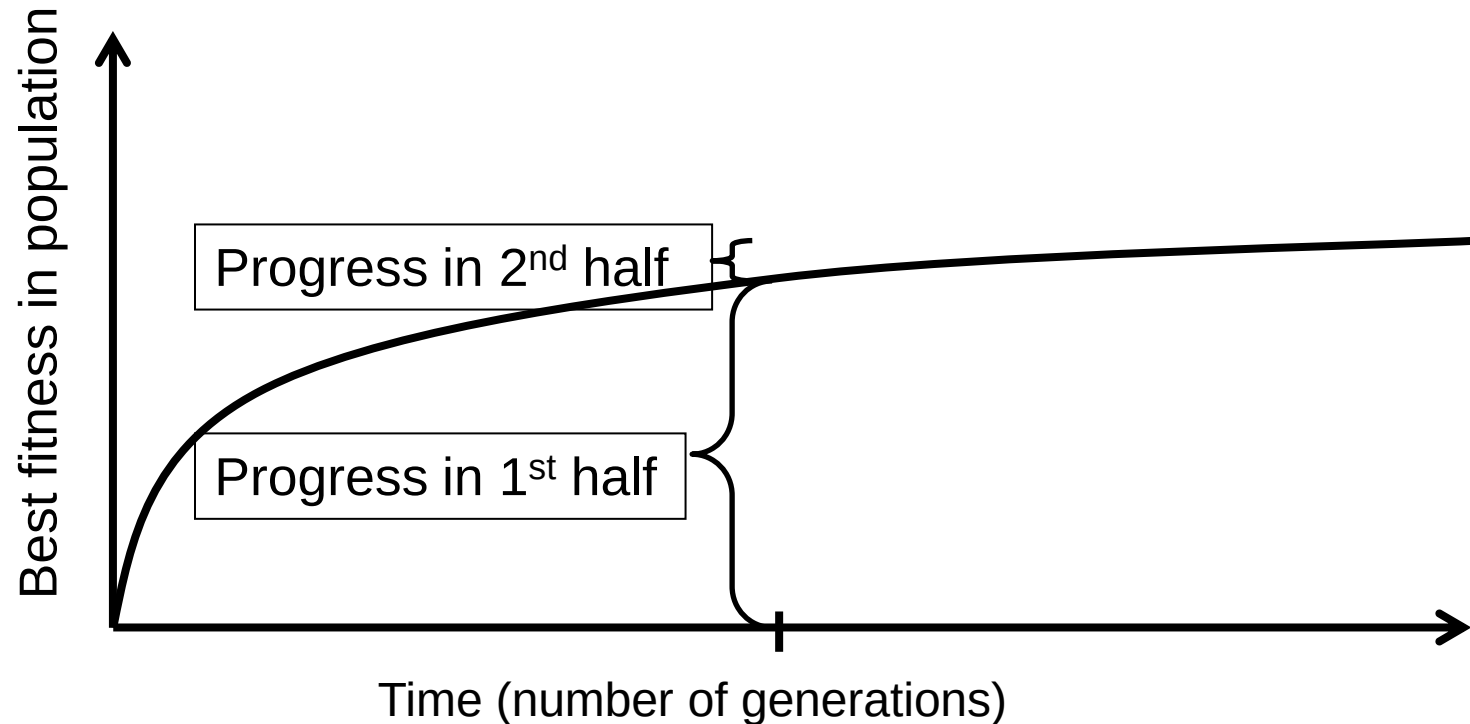
# Best Solution (Distance = 420)



# Overview of Performance

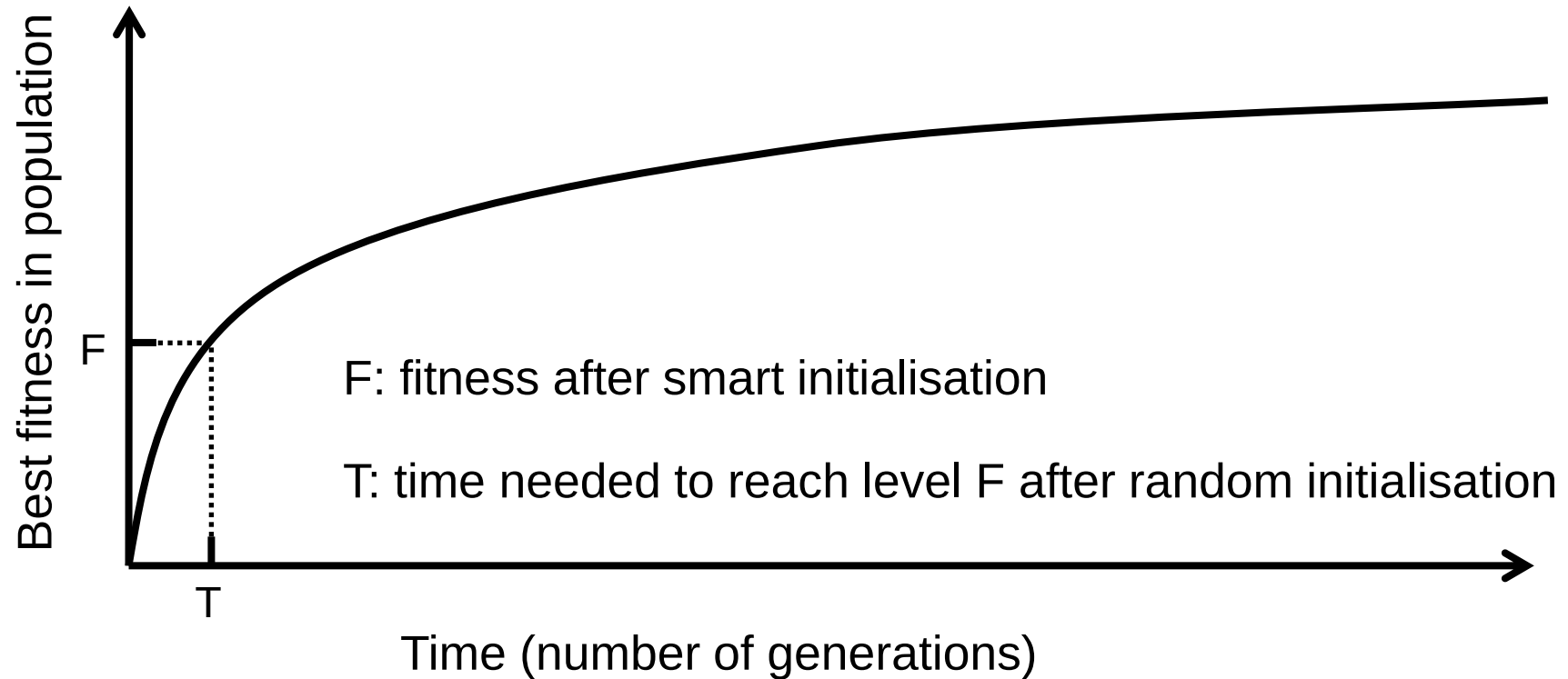


# Are long runs beneficial?



- Answer:
  - it depends how much you want the last bit of progress
  - it may be better to do more shorter runs

# Is it worth expending effort on smart initialisation?



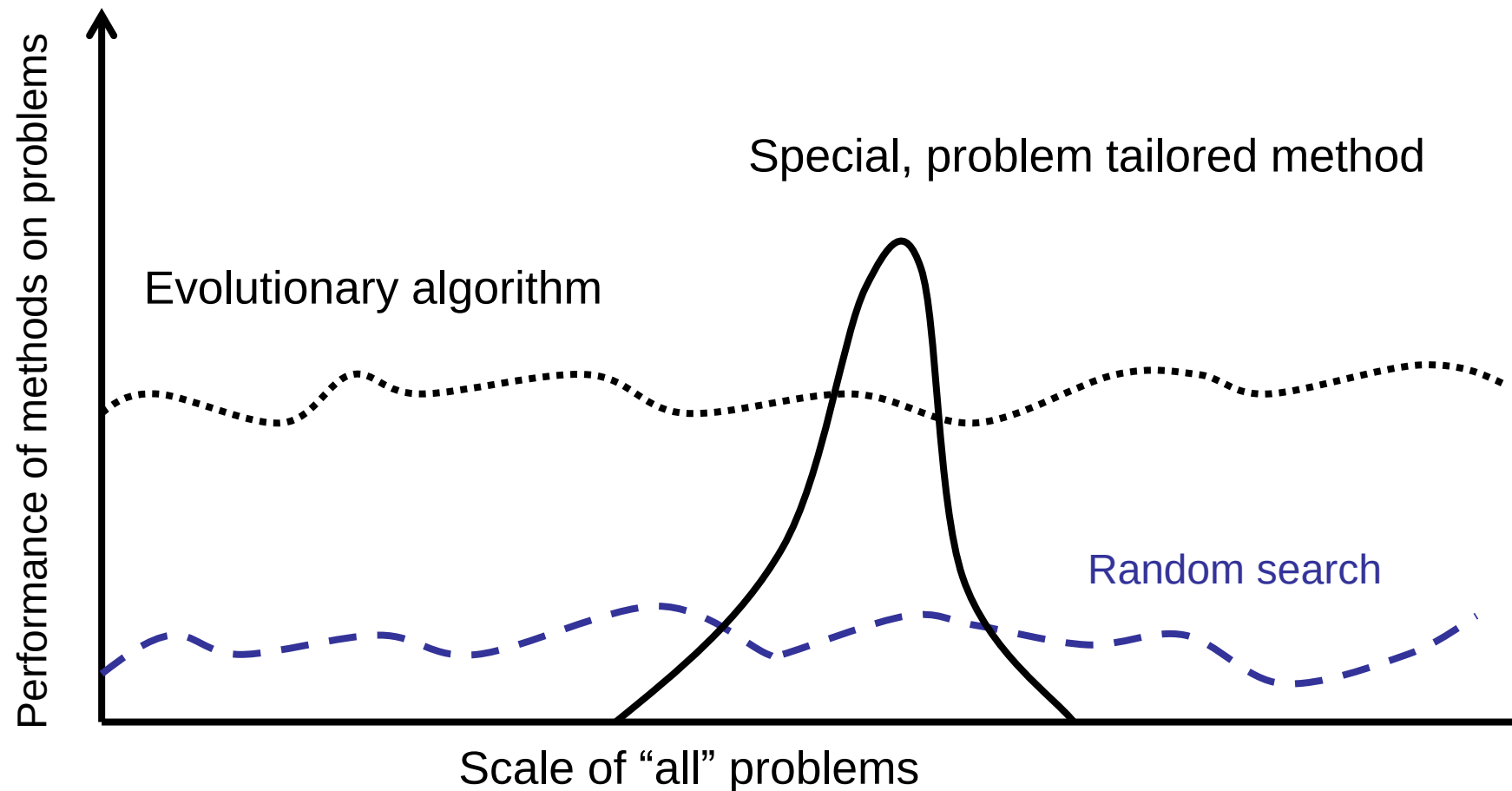
- Answer : it depends:
  - possibly, if good solutions/methods exist.
  - care is needed, see chapter on hybridisation

# Evolutionary Algorithms in Context

---

- There are many views on the use of EAs as robust problem solving tools
- For most problems a problem-specific tool may:
  - perform better than a generic search algorithm on most instances,
  - have limited utility,
  - not do well on all instances
- Goal is to provide robust tools that provide:
  - evenly good performance
  - over a range of problems and instances

# EAs as problem solvers: Goldberg's 1989 view



# EAs and domain knowledge

---

- Trend in the 90's:  
adding problem specific knowledge to EAs  
(special variation operators, repair, etc)
- Result: EA performance curve “deformation”:
  - better on problems of the given type
  - worse on problems different from given type
  - amount of added knowledge is variable
- Recent theory suggests the search for an “all-purpose” algorithm may be fruitless

# EC and Global Optimisation

---

- Global Optimisation: search for finding best solution  $x^*$  out of some fixed set  $S$
- Deterministic approaches
  - e.g. box decomposition (branch and bound etc)
  - Guarantee to find  $x^*$ , but may run in super-polynomial time
- Heuristic Approaches (generate and test)
  - rules for deciding which  $x \in S$  to generate next
  - no guarantees that best solutions found are globally optimal

# EC and Neighbourhood Search

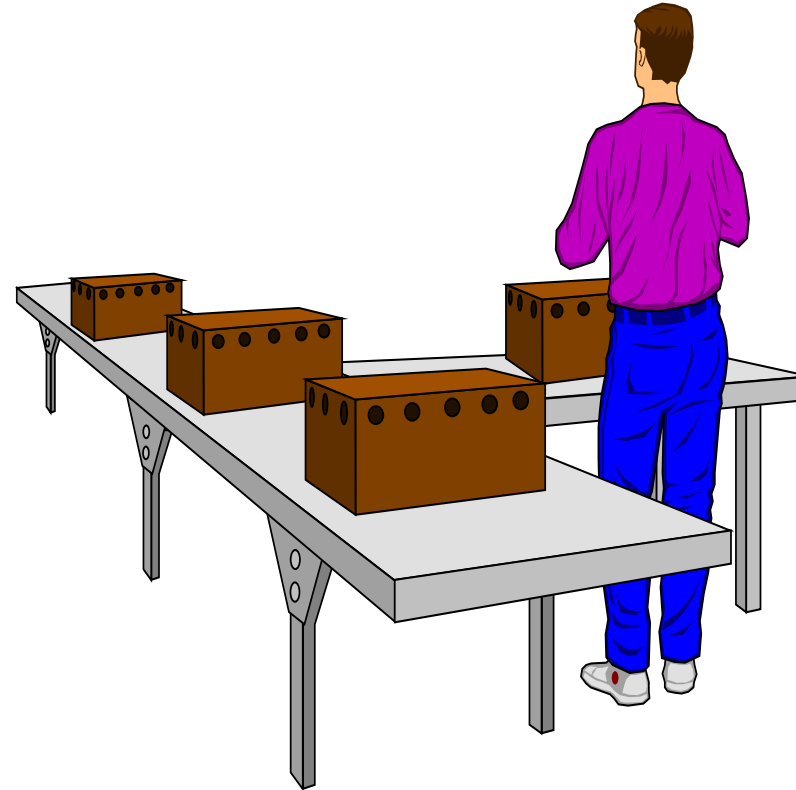
---

- Many heuristics impose a neighbourhood structure on  $S$
- Such heuristics may guarantee that best point found is *locally optimal* e.g. Hill-Climbers:
  - **But** problems often exhibit many local optima
  - Often very quick to identify good solutions
- EAs are distinguished by:
  - Use of population,
  - Use of multiple, stochastic search operators
  - Especially variation operators with arity  $>1$
  - Stochastic selection

# Considering the GA Technology

*“Almost eight years ago ... people at Microsoft wrote a program [that] uses some genetic things for finding short code sequences. Windows 2.0 and 3.2, NT, and almost all Microsoft applications products have shipped with pieces of code created by that system.”*

- Nathan Myhrvold, Microsoft Advanced Technology Group,  
*Wired*, September 1995



# Benefits of Genetic Algorithms

---

- Concept is easy to understand
- Modular, separate from application
- Supports multi-objective optimization
- Good for “noisy” environments
- Always an answer; answer gets better with time
- Inherently parallel; easily distributed

## Benefits of Genetic Algorithms (cont.)

---

- Many ways to speed up and improve a GA-based application as knowledge about problem domain is gained
- Easy to exploit previous or alternate solutions
- Flexible building blocks for hybrid applications
- Substantial history and range of use

# When to Use a GA

---

- Alternate solutions are too slow or overly complicated
- Need an exploratory tool to examine new approaches
- Problem is similar to one that has already been successfully solved by using a GA
- Want to hybridize with an existing solution
- Benefits of the GA technology meet key problem requirements

# Some GA Application Types

| Domain                     | Application Types                                                                         |
|----------------------------|-------------------------------------------------------------------------------------------|
| Control                    | gas pipeline, pole balancing, missile evasion, pursuit                                    |
| Design                     | semiconductor layout, aircraft design, keyboard configuration, communication networks     |
| Scheduling                 | manufacturing, facility scheduling, resource allocation                                   |
| Robotics                   | trajectory planning                                                                       |
| Machine Learning           | designing neural networks, improving classification algorithms, classifier systems        |
| Signal Processing          | filter design                                                                             |
| Game Playing               | poker, checkers, prisoner's dilemma                                                       |
| Combinatorial Optimization | set covering, travelling salesman, routing, bin packing, graph colouring and partitioning |

# Quiz

---

Let  $F_1, \dots, F_n$  are features of a classification problem. Find  $k$  best features to get the best results for a given classifier (e.g. for an ELM network). Propose a genetic algorithm.

# Questions ?

---

